

Weather Analysis

Class WeatherAnalysis (Main class)

→ Class WeatherMapper (Sub-classes)
→ Class WeatherReducer
→ Static Main Method.

① Static main method

⇒ Configuration conf = new Configuration();

* It creates Hadoop Configuration object;

→ It contains
- Hadoop's environment variables.
- job parameters
- other settings.

⇒ Job job = Job.getInstance(conf, "Weather Analysis")

↓
It creates a MapReduce job object with the specified configuration.

↓
It's a job name, helpful for identifying the job on the Hadoop cluster.

Setting Jar File

⇒ job.setJarByClass(WeatherAnalysis.class)

* Makes sure Hadoop knows which jar to distribute.

* This instruction ensures that jar file is sent to all the nodes by Hadoop.

* This makes sure all cluster nodes can access Mapper and Reducer classes.

* It groups bundles the mapper, reducer and other classes into a single jar file.

⇒ Setting the mapper and reducer class

```
job.setMapperClass(WeatherMapper.class);
```

```
job.setReducerClass(WeatherReducer.class);
```

* It specifies the mapper and reducer classes to use in this job.

* WeatherMapper → processes input and emits intermediate key-value pairs

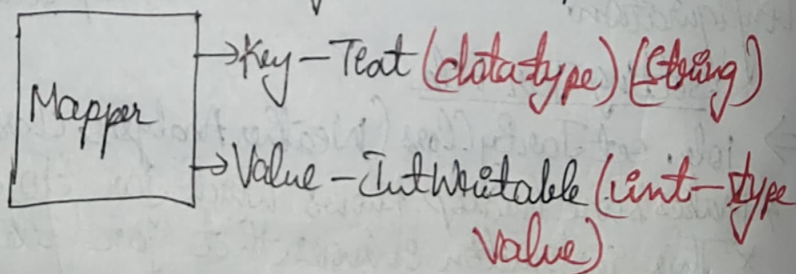
WeatherReducer → Aggregates and processes the mapper output.

⇒ Setting the map output key-value types.

```
job.setMapOutputKeyClass(Text.class);
```

```
job.setMapOutputValueClass(IntWritable.class);
```

* It defines output data types from the mapper

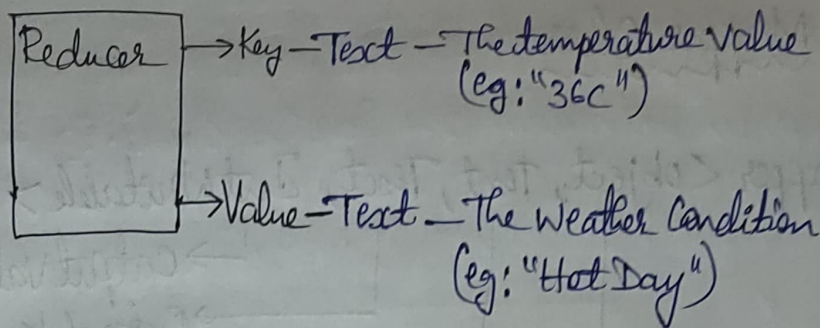


⇒ Setting Final output key-value types

```
job.setOutputKeyClass(Text.class)
```

```
job.setOutputValueClass(Text.class)
```

* defines the final output data types emitted by the reducer.



⇒ Specifying input and output paths

`FileInputFormat.addInputPath(job, new Path(args[0]));`

`FileOutputFormat.setOutputPath(job, new Path(args[1]));`

`args[0]` → reads the path specified

→ This where the input file or directory located.

→ input path.

`args[1]` → It writes the output to `args[1]`

→ This output directory must not be present already.

⇒ Submitting the job

`System.exit(job.waitForCompletion(true)? 0: 1);`

→ waits until the job finishes.

↳ Submits the job and waits until it finishes.

→ It will terminate the program with a proper exit status.

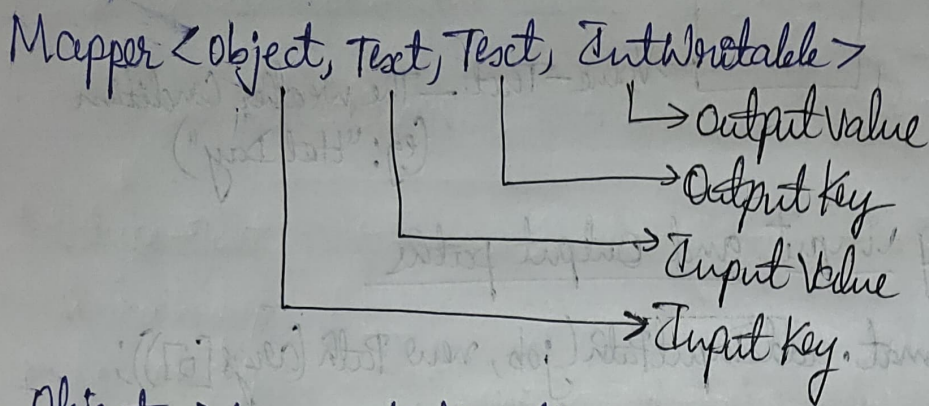
0 → Success (Job Completed without errors)

1 → Fail (Job did not complete successfully)

* Submits the ^{Mapreduce} job to the hadoop Job Tracker (YARN Resource Manager)

* False → Submits the job, but does not wait.

⑤ Mapper Class



Object → represents the offset (byte position) of the line in the input file.

Ex:-

0	1	2
3	0	ln
3	4	5
2	5	ln
6	7	8
4	0	ln
9	10	11
1	8	

30 → offset = 0

25 → offset = 3

40 → offset = 6

18 → offset = 9.

* Hadoop processes large files in a distributed manner.

* Each Mapper knows where to start reading based on the offset.

* If a job fails or restarts, Hadoop can resume from a specific byte offset, avoiding reprocessing the entire file.

Text → represents the actual input data (each line of text)

→ Each line contains temperature value.

Text → Represents the key emitted by the mapper
→ it is a String key "day type".

IntWritable → Represents value emitted by the mapper
→ it is the temperature as an integer.

③ public void map(Object key, Text value, Context context)

Key → Represents byte offset of the line (where the line starts in the file)
→ 0, 3, 6, 9

Value → Actual content of the input line
→ "30", "25", "40", "18"

Context → used to write key-value output and interact with the Hadoop system.

1st call → map(0, "30", context)

2nd call → map(3, "25", context)

3rd call → map(6, "40", context)

4th call → map(9, "18", context)

Roles of Context object:-

1) Write output key-value pairs using:

context.write(new Text("Day Type"), new IntWritable(50));

2) Report progress to Hadoop (used in large data sets)

3) Write custom counters and status updates.

④ `String[] fields = value.toString().split(",");`

* helps in extracting the specific fields. Ex: 24-03-2025, 30°C
24th
1st

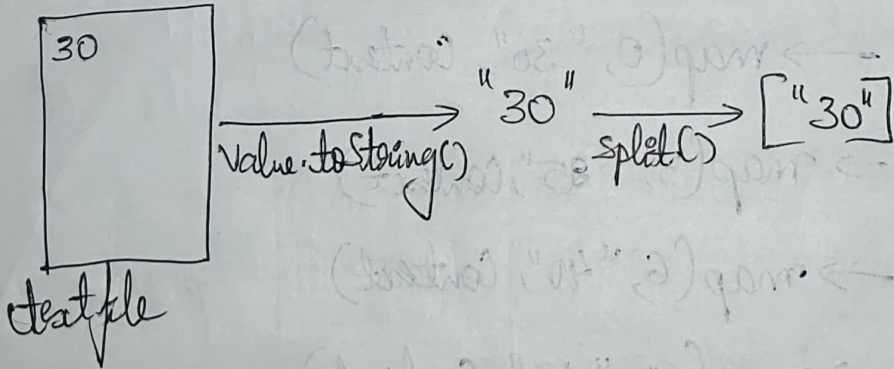
Value.toString()

dotatype - Test,
which is a hadoop
specific dotatype.

- Converts that object into a Java String
- Without this conversion, split() cannot be performed, because it is a String operation.

- $\text{split}(u, v)$

- It is a String method in Java that breaks the String into an array of substrings wherever it finds a comma (,).
- resulting substrings are stored in a String array called fields.



⑤ if (fields.length == 1)

- * It ensures only valid lines with one value are processed.

- * To avoid errors when the input line is empty or incorrectly formatted.

* If fields.length $\neq 1$, it means the data is invalid, or

has missing values.

⑥ Context.write()

→ It is a method to emit Key-Value pair.

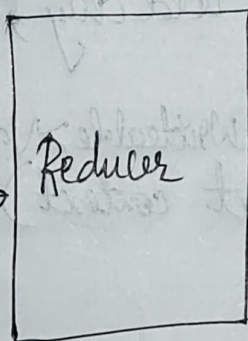
→ Mappers produce intermediate Key-value pairs. These pairs are grouped by keys and sent to the reducer.

→ It is an object provided by Hadoop's MapReduce framework.

→ It allows mapper/reducer to write Key-value pairs.

→ It also reports the status and progress of the job.

("DayType", [30, 25, 40, 18])



* using a constant key "DayType", all the temperature values are grouped together.

("DayType", [30, 25, 40, 18])

* This makes it easy to perform aggregation or analysis.

⑦ Reducer Class

Reducer < Text, IntWritable, Text, Text >

Text → Key - in

→ data type - Hadoop Text

→ Key emitted by a mapper (eg: "DayType")

IntWritable → Value-In

→ Hadoop IntWritable-data type

→ Value emitted by the mapper
(eg., temperature values)

Text → Key-Out

→ Hadoop text-data type

→ Key emitted by the Reducer after aggregation
(eg:- temperature in "C")

30C, 40C, 50C

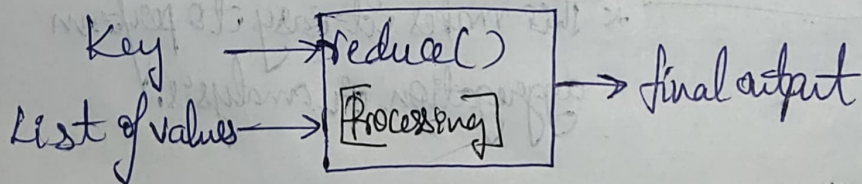
Text → Value-Out

→ Hadoop text-data type

→ Value emitted by the reducer

(eg:- "Hot day", "Cold day", etc)

⑧ reduce(Text key, Iterable<IntWritable> values,
Context context)



Text Key → it represents grouping key from mapper output

→ All values associated with this key, are sent together to one reducer call.

→ Ex:- Key → "DayType"

Iterable <Writable> Values.

→ This is a Collection (like a list) of all values associated with the key.

→ temperature values (eg: -35, 25, 40, 18)

Context Context → used to write output and report progress

→ used to emit final key-value pairs.

reduce("DayType", [30, 25, 40, 18], context)